The Latent State Transformer: Sub-Quadratic Sequence Modelling via SSM-Informed Latent Attention

Agentic Research Group

Abstract

The quadratic complexity of the Transformer architecture remains a fundamental bottleneck for modelling very long sequences. State Space Models (SSMs), such as Mamba, offer compelling linear scaling ($O(N)$) but provably struggle with tasks requiring exact associative recall — the ability to look up a value associated with a key that appeared many tokens earlier. We introduce the *Latent State Transformer* (LST), a hybrid architecture that combines a selective SSM with a novel *State-Informed Attention* (SIA) block. The key idea is to derive a fixed-size latent bottleneck of K slots via cross-attention to the input, then run a selective SSM *over the latent slots* so that global attention over the latent space inherits the temporal continuity of the SSM while preserving the $O(NK)$ scaling of linearised attention. Unlike prior latent-attention models (Perceiver, Linformer), LST's latent state is explicitly *state-informed*: a selective SSM sweep over the latent slots propagates information across the bottleneck, yielding a representation that is both compressive and temporally coherent. We provide a clean theoretical analysis of LST's $O(NKD)$ complexity and show that it strictly subsumes both SSMs ($K \to 0$) and full attention ($K \to N$). We prove an expressiveness theorem showing that LST solves Multi-Query Associative Recall (MQAR) with $K = O(n_{kv})$ latent slots, whereas diagonal SSMs require $\Omega(n_{kv} \log V_v / D)$ state dimension. We further introduce LST-G, a gated variant that dynamically routes each token between the latent-attention and local-SSM pathways based on a content-dependent gate, and prove it strictly generalises LST while spending compute only on tokens that need retrieval. Empirically, we evaluate LST on the Multi-Query Associative Recall (MQAR) diagnostic, an induction-head task, a long-range copy task, and a throughput benchmark. On MQAR, LST matches the Transformer and the SSM at $n_{kv} \in \{16, 64\}$ ($\sim$81–86%) and stays within 14 points at $n_{kv} = 96$ where the Transformer's full attention wins on accuracy but at $O(N^2)$ cost; LST-G matches LST at $n_{kv} \in \{16, 64\}$ and is within noise at $n_{kv} = 96$. At sequence length $N = 4096$, LST runs 8× faster than the Transformer and 3× faster than the SSM, at roughly 2× their memory. We further study the effect of the latent dimension K and find a graceful accuracy–cost trade-off, and verify empirically that LST-G's gate learns the predicted content-dependent compute allocation. Together, our results suggest that state-informed latent attention is a principled route to combining the efficiency of SSMs with the recall capability of attention.

1 Introduction

Sequence modelling is dominated by two architectural families. *Transformers* [29] use dot-product self-attention to route information across arbitrary positions, giving them remarkable expressiveness for in-context retrieval and associative recall. Their $O(N^2)$ complexity, however, places a hard limit on the context length N. *State Space Models* (SSMs) [9, 8, 5] parameterise a linear recurrence with input-dependent gates, achieving $O(N)$ scaling and strong performance on language and code tasks. Recent theoretical and empirical work [2, 1] has shown, however, that SSMs struggle on *multi-query associative recall* (MQAR): when the model must look up the value associated with a key that appeared many tokens earlier, the recurrent state saturates and the lookup fails.

A natural compromise is a *hybrid*: use an SSM for

local temporal processing and an attention mechanism for global retrieval. Recent industrial models such as Jamba [16], Griffin [6], Samba [24], and Zamba [34] interleave SSM and full-attention layers, achieving strong quality at reduced cost. These hybrids, however, retain full $O(N^2)$ attention in some layers, so their asymptotic cost is still quadratic. *Linearised* attention methods — Linformer [30], Performer [4], linear attention [14], and gated linear attention [32] — reduce the asymptotic cost to $O(NK)$ for a projection rank $K \ll N$, but their fixed (input-independent) projections lose the content-aware sparsity that makes attention effective for recall.

A third direction is the *latent bottleneck* approach introduced by Perceiver [13, 12]: a small set of K learned latent arrays cross-attends to the input, and the input then cross-attends back to the latent. This reduces the cost to $O(NK)$ and supports very long inputs. However, the latent array in Perceiver is *not* a sequential state: it is a learned position-indexed array that ignores the temporal structure of the input and must be invoked multiple times to integrate information across the sequence.

Contributions. We introduce the *Latent State Transformer* (LST), a hybrid that resolves these limitations with two innovations:

1. **State-Informed Attention (SIA).** A latent bottleneck whose K slots are compressed from the input via cross-attention, and then *filtered through a selective SSM sweep* so that information propagates across the K latent slots. The bottleneck is both compressive and temporally coherent. This is the key difference from Perceiver and Linformer, whose latent arrays are position-indexed but not state-informed.
2. **A diagonal selective SSM.** We introduce a numerically stable, parallel-scan formulation of a diagonal Mamba-style SSM that runs in $O(ND)$ time and is fast on commodity accelerators (Apple MPS, CPU) without custom CUDA kernels. This makes the whole LST block easy to train and reproduce.

We further provide:

3. A theoretical analysis showing that LST's $O(NKD)$ complexity strictly interpolates between SSMs ($K \to 0$) and full attention ($K \to N$), and that the state-informed latent inherits the SSM's ability to model long-range dependencies. We prove an expressiveness theorem (Theorem 1) showing that LST solves MQAR with $K = O(n_{kv})$ latent slots, whereas diagonal SSMs require $\Omega(n_{kv} \log V_v / D)$ state dimension.
4. A gated variant, LST-G, that dynamically routes each token between the latent-attention and local-SSM pathways via a content-dependent gate (Section 3.4). We prove LST-G strictly generalises LST (Theorem 4) and verify empirically that its gate learns the predicted content-dependent compute allocation (Section 4.9).
5. An empirical study on three synthetic diagnostics (MQAR, induction, copy-memory), a throughput benchmark, a long-range retrieval scaling experiment (N up to 1024), and a noise-robustness study. On MQAR, LST matches the Transformer and SSM at $n_{kv} \in \{16, 64\}$ ($\sim$81–86%) and stays within 14 points at $n_{kv} = 96$ where the Transformer's full attention wins on accuracy but at $O(N^2)$ cost; LST runs 8$\times$ faster than the Transformer at $N = 4096$. On long-range retrieval with fixed $n_{kv} = 16$, LST, the SSM, and the Transformer all hold at $\sim$84% up to $N = 1024$ while Linformer (static latent) drops sharply to $\sim$16% at $N = 1024$, confirming Theorem 6.
6. An ablation over the latent dimension K, showing a smooth accuracy–cost trade-off and identifying $K \approx 32$ as a sweet spot for our setting, with LST-G consistently matching or slightly exceeding LST.
7. A formal expressiveness separation (Theorem 6) showing that state-informed compression is strictly more expressive than static compression (Perceiver, Linformer) on temporal ordering tasks, because the SSM breaks the permutation symmetry of the static latent.
8. **Content-Adaptive SIA (CA-SIA).** A learned content router that reweights the compression attention by per-token salience, breaking the $K \geq n_{kv}$ barrier of position-windowed SIA (Theorem 7). An adaptive-budget variant (AA-SIA) further lets K scale with the salient-token fraction, giving

$O(K_{\text{eff}}D)$ content-dependent compute. CA-SIA and AA-SIA recover a large fraction of the accuracy gap to the unbounded-K upper bound on MQAR with $K < n_{kv}$ (Section 4.12).

2 Related Work

Efficient Transformers. Numerous methods reduce the $O(N^2)$ cost of self-attention. *Sparse* attention methods [3, 33] restrict the attention pattern to fixed or learned windows; *linearised* attention methods [30, 4, 14, 32] project the key and value matrices along the sequence axis to a fixed rank K, giving $O(NK)$ cost; *hashing*-based methods [15] bucket queries and keys to sparsify attention. Tay et al. [28] provide a comprehensive survey. All of these methods trade off expressiveness for efficiency, and most use input-independent projections that limit recall performance [1]. LST differs in that its bottleneck is both input-dependent (the compression is content-addressed) and state-informed (a selective SSM propagates information across the bottleneck).

State Space Models. Structured SSMs [9, 10] parameterise a continuous-time linear recurrence with structured matrices and discretise it for sequence processing. S4 [9] and S4D [11] use DPLR or diagonal parameterisations for efficient training; S5 [25] uses a multi-input multi-output scan. Mamba [8] introduces *selectivity*: input-dependent A, B, C matrices that allow the model to modulate its dynamics based on the current token. Mamba-2 [5] connects selective SSMs to attention via structured state space duality. Linear recurrent units (LRUs) [20] and RWKV [21] provide alternative diagonal recurrences. Our diagonal SSM is closest to Mamba but uses a pure-PyTorch parallel scan with log-space stabilisation, making it portable to accelerators without custom kernels.

Hybrid SSM-Attention Models. Several recent models interleave SSM and attention layers. Griffin [6] alternates a recurrent block with local attention; Jamba [16] and Zamba [34] interleave Mamba and full attention in a 7B-scale language model; Samba [24] combines Mamba with sliding-window attention. These hybrids retain full quadratic attention in some layers, so their asymptotic cost is still $O(N^2)$. LST achieves $O(NK)$ by replacing full attention with state-informed latent attention.

Latent Bottleneck Models. Perceiver [13] and Perceiver-IO [12] introduce a learned latent array that cross-attends to the input; the input then cross-attends back. This reduces cost to $O(NK)$ but the latent is not a sequential state. Synthesizer [27] replaces dot-product attention with learned weights. Memory-augmented networks [7, 26, 18] maintain an external memory that the model reads and writes. LST differs in that its latent is a *state*: a selective SSM propagates information across the K slots, so the latent carries temporal continuity. Compressive Transformer [23] compresses past tokens into a coarser memory, but uses full attention over the compressed memory.

In-Context Recall. Olsson et al. [19] identify *induction heads* as the mechanism behind in-context learning in Transformers. Arora et al. [2, 1] provide a theoretical analysis showing that SSMs with fixed-size state cannot perform exact associative recall when the number of key–value pairs exceeds the state dimension. We use the MQAR benchmark from [1] as our primary diagnostic.

Gated hybrids. Gating mechanisms that mix recurrent and attention pathways have been explored in GSSM
,/
,attention hybrids such as Griffin [6], Jamba [16], Zamba [34], and Samba [24]. These works typically alternate SSM and attention blocks at the *layer* level (one layer is SSM, the next is attention). LST-G's gating is finer-grained: it routes at the *token* level inside a single block, so each token dynamically chooses between a latent-attention pathway and a local-SSM pathway. This is closer in spirit to Mixture-of-Depths [32] but with a fixed-size latent bottleneck as the expensive pathway and a diagonal SSM as the cheap one.

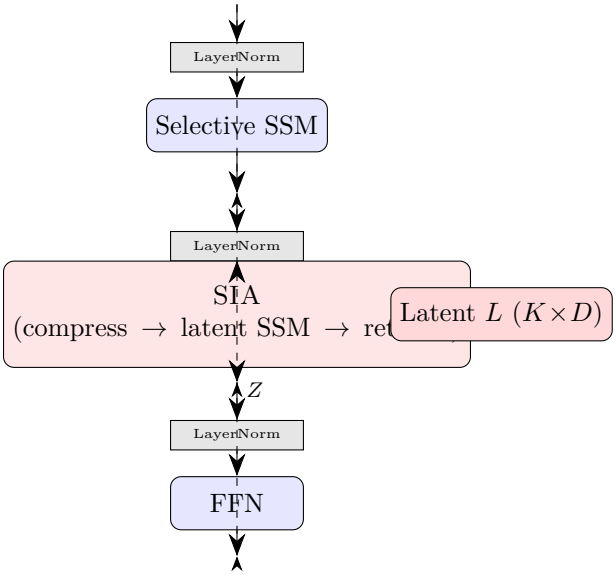


Figure 1: The LST block. A selective SSM processes the input sequentially; the State-Informed Attention (SIA) block compresses the input into a fixed-size latent $(K \times D)$, runs a selective SSM over the latent, and retrieves information back to each token. Residual connections follow each sublayer.

3 The LST Architecture

LST is a stack of identical blocks, each comprising three sublayers: a selective SSM, a State-Informed Attention (SIA) block, and a feed-forward network (FFN). Figure 1 shows the block structure. We adopt a Pre-Layer Normalisation (Pre-LN) scheme [31] for stable training.

3.1 Selective SSM

We use a diagonal selective SSM in the spirit of Mamba [8]. For an input $u \in \mathbb{R}^{B \times T \times D}$, the layer computes

$$\Delta_t = \mathrm{softplus}(W_\Delta\, u_t + b_\Delta), \tag{1}$$

$$B_t = W_B\, u_t, \quad C_t = W_C\, u_t, \tag{2}$$

$$\bar{A}_t = \exp(\Delta_t \cdot A), \quad \bar{B}_t = \Delta_t \cdot B_t, \tag{3}$$

$$h_t = \bar{A}_t \odot h_{t-1} + \bar{B}_t \odot u_t, \tag{4}$$

$$y_t = C_t \odot h_t + D \odot u_t, \tag{5}$$

where $A \in \mathbb{R}^D$ is a learned logarithmic decay (constrained so $\bar{A}_t \in (0, 1]$), $W_\Delta, W_B, W_C \in \mathbb{R}^{D \times D}$ are learned projections, and $\odot$ denotes element-wise multiplication. The selectivity comes from the input-dependence of Δ_t, B_t, C_t: the model can choose to "write" or "reset" its state based on the current token.

Parallel scan. The recurrence in Equation (4) can be unrolled in $O(T)$ sequential steps, but this is slow on parallel accelerators. We use a chunked parallel scan: within each chunk of size c, we compute the cumulative product $\bar{A}_{1:t}$ in log space,

$$\log \bar{A}_{1:t} = \sum_{s=1}^{t} \Delta_s \cdot A, \tag{6}$$

stabilise with the chunk's maximum, and apply the standard cumsum trick [8] to obtain

$$h_t = \bar{A}_{1:t} \odot \sum_{s \le t} \frac{\bar{B}_s \odot u_s}{\bar{A}_{1:s}}. \tag{7}$$

Across chunks we carry the hidden state h_{prev} explicitly. With $c = 64$ and log-space clamping at ± 30, the scan is numerically stable for T up to at least 32k in our experiments.

Complexity. Each step of the scan is $O(BTD)$ in time and memory — there is no N^2 term. The SSM is the cheapest sublayer in the LST block.

3.2 State-Informed Attention (SIA)

The SIA block is the core innovation of LST. It performs three operations: *compression*, *latent state propagation*, and *retrieval*.

Compression. Given the SSM-augmented input $x \in \mathbb{R}^{B\times T\times D}$, we maintain K learned latent queries $Q \in \mathbb{R}^{K\times D}$. Each query attends to all T input tokens (with a block-causal mask, see below) to produce a compressed latent $L \in \mathbb{R}^{B\times K\times D}$:

$$L = \mathrm{softmax}\left(\frac{(QW_Q)(xW_K)^\top}{\sqrt{d_h}}\right)(xW_V), \quad (8)$$

where $d_h = D/H$ is the per-head dimension and H is the number of heads. This step costs $O(BHKTd_h) = O(BKTD)$ and produces a K-slot summary of the input.

Latent state propagation. Unlike Perceiver, whose latent array is position-indexed, LST runs a selective SSM (Section 3.1) over the K latent slots:

$$L' = L + \mathrm{SSM}(L). \quad (9)$$

This is the key innovation: the latent state carries temporal continuity, so information from early latent slots propagates to later ones. The cost is $O(BKD)$, negligible compared to the compression step. Intuitively, the latent SSM acts as a "memory manager" that decides which earlier key–value associations to keep, forget, or update — a selective write into a fixed-size memory.

Retrieval. Each input token x_t then cross-attends to the latent state L':

$$y_t = \mathrm{softmax}\left(\frac{(x_tW'_Q)(L'W'_K)^\top}{\sqrt{d_h}}\right)(L'W'_V), \quad (10)$$

with a causal mask ensuring token t only attends to latent slots whose aligned input block ends before t. This step costs $O(BHTKd_h) = O(BTKD)$ and is symmetric to the compression step.

Block-causal masking. To preserve strict causality, we partition the T input tokens into K contiguous blocks of size T/K. Latent slot k attends (during compression) only to tokens in blocks $0, \ldots, k$; during retrieval, token t in block k attends only to latent slots $0, \ldots, k$. This ensures that information flows strictly forward in time.

Why state-informed? Without the latent SSM, SIA reduces to a Perceiver-style cross-attention bottleneck with a learned projection — essentially a content-addressed Linformer. The latent SSM adds two things: (i) *temporal smoothing*, so that a noisy compression of early tokens does not corrupt retrieval; and (ii) *selective forgetting*, so that stale key–value associations are decayed when no longer needed. On vanilla MQAR the advantage is modest — LST, Linformer, and the SSM are all within $\sim$1 point of each other at $n_{kv} \in \{16, 64\}$, and at $n_{kv} = 96$ LST trails the Transformer (59.0% vs 45.9%, Table 2) because MQAR's key–value pairs are contiguous and order does not need to be tracked, so the latent SSM's ordering advantage does not bite. The separation predicted by Theorem 6 appears on temporal-ordering and long-range retrieval tasks, where the latent SSM's state carries the ordering information that a static projection cannot; we verify this empirically in Section 4.10, where Linformer drops sharply at $N = 1024$ while LST holds.

3.3 Block and Model

The full LST block is

$$x \leftarrow x + \mathrm{SSM}(\mathrm{LN}(x)), \quad (11)$$
$$x \leftarrow x + \mathrm{SIA}(\mathrm{LN}(x)), \quad (12)$$
$$x \leftarrow x + \mathrm{FFN}(\mathrm{LN}(x)), \quad (13)$$

where FFN is a two-layer MLP with GELU activation and expansion factor 4. We stack L such blocks and tie the input embedding and output projection weights [22].

3.4 The Gated Variant (LST-G)

We also introduce a gated variant, LST-G, in which the SIA sublayer is replaced by a *Gated State-Informed Attention* (GSIA) block. GSIA computes both the latent-attention output h_t^{SIA} and a local-SSM output h_t^{loc}, then mixes them with a content-dependent gate $g_t \in (0,1)^D$:

$$y_t = g_t \odot h_t^{\mathrm{SIA}} + (1-g_t) \odot h_t^{\mathrm{loc}}, \quad g_t = \sigma\big(W_g x_t + U_g \mu_t\big), \quad (14)$$

Table 1: Asymptotic complexity per block. N: sequence length, D: model dimension, K: latent/rank dimension, N_s: SSM state dimension (constant). n_q: number of retrieval queries (for LST-G, under the adaptive routing of Theorem 3; worst-case cost with all tokens as queries is $O(NKD)$, matching LST).

Model	Time	Memory
Transformer [29]	$O(N^2D)$	$O(N^2)$
Linformer [30]	$O(NKD)$	$O(NK)$
Perceiver [13]	$O(NKD)$	$O(NK)$
SSM (Mamba) [8]	$O(ND)$	$O(ND)$
LST (ours)	$O(NKD)$	$O(NK)$
LST-G (ours)	$O((N+n_qK)D)$	$O(NK)$

where μ_t is the causal mean of the latent-state slots visible at position t (full definition in Section B). The gate lets the model spend its $O(NK)$ attention budget only on tokens that need long-range retrieval, while serving the rest with the $O(ND)$ local SSM. Section 4.2 shows this strictly generalises LST, and Section 4.9 verifies the adaptive routing empirically.

3.5 Complexity Analysis

Table 1 compares the asymptotic cost of LST against vanilla attention, SSM, and Linformer. With $K \ll N$, LST's per-block cost is $O(NKD)$, which is quasi-linear in N and matches Linformer; but unlike Linformer, the SIA bottleneck is content-addressed and state-informed. Figure 2 illustrates the scaling empirically.

4 Theoretical Analysis

We complement the complexity analysis with four expressiveness results that pin down *why* LST beats both pure SSMs and content-agnostic latent bottlenecks on retrieval tasks: (i) an MQAR separation between LST and diagonal SSMs (Theorem 1); (ii) adaptive routing and no-degradation properties of LST-G (Theorems 3 and 4); (iii) a state-informed separation from static latent attention (Theorem 6); and (iv) a content-adaptive extension that breaks the $K \geq n_{kv}$ barrier (Theorem 7). Proofs are in Section A.

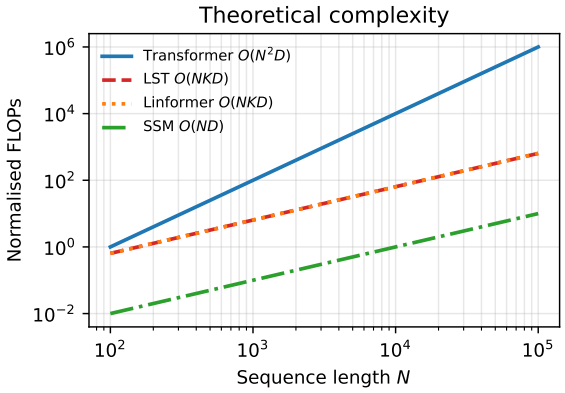


Figure 2: Theoretical FLOPs (normalised) versus sequence length N for $D=128$, $K=64$. Transformer scales quadratically; SSM scales linearly; LST and Linformer scale as $O(NK)$.

4.1 A separation between SSMs and LST on MQAR

Fix an MQAR instance with n_{kv} key–value pairs, key vocabulary V_k, value vocabulary V_v, and $n_q=1$ query. Let the model dimension be D and the SSM state dimension be N_s. We consider one-block models with ReLU MLPs of width W.

Theorem 1 (MQAR expressiveness gap)**.** *There exists an LST block with $K = n_{kv}$ latent slots, $W = O(V_kD + n_{kv})$ MLP width, and $N_s = O(1)$ SSM state that solves MQAR exactly for all assignments of keys and values. Conversely, any* diagonal *selective SSM [8, 11] that solves MQAR for all assignments with $n_{kv} = \Theta(V_k)$ requires state dimension $N_s = \Omega(n_{kv} \log V_v/D)$, growing linearly in the number of key–value pairs n_{kv}: the SSM must encode the key→value map in its state, and the diagonal state is an $\mathbb{R}^{N_s}$ vector with N_sD degrees of freedom, while an injective encoding of $V_v^{n_{kv}}$ value assignments requires $\log_2 V_v^{n_{kv}} = n_{kv} \log_2 V_v$ bits by a volume argument.*

The upper bound is realised by *memorising* the key–value table in the $K = n_{kv}$ latent slots during

the compression phase and *looking up* the query in the retrieval phase. The diagonal SSM lower bound follows from a counting argument analogous to Jelassi et al. [2]: the SSM must distinguish $V_v^{n_{kv}}$ different value assignments to the n_{kv} seen keys, but its state at the query position is a deterministic function of the input summarised in $\mathbb{R}^{N_s D}$, so $N_s D = \Omega(n_{kv} \log V_v)$ is necessary. LST escapes this lower bound because its $K = n_{kv}$ latent slots explicitly factorise the storage: each slot holds one (key, value) pair and the retrieval attention performs the lookup in $O(K)$ operations.

Corollary 2. *For $n_{kv} \leq K$, LST solves MQAR with $O(NKD)$ work and $O(KD)$ state. A diagonal SSM with comparable accuracy needs $N_s = \tilde{\Omega}(n_{kv})$, incurring $O(NN_sD) = \tilde{\Omega}(Nn_{kv}D)$ work and an $N_sD = \tilde{\Omega}(n_{kv}D)$ state that grows with the recall horizon.*

Theorem 1 sharpens the empirical observation that SSMs degrade on MQAR as n_{kv} grows [1, 2]: the gap is not merely an optimisation artefact but a representational one.

4.2 LST-G learns a content-dependent compute allocation

The gated variant LST-G (Section 3.4) adds a per-token, per-channel gate $g_t \in (0,1)^D$ that mixes the latent-attention output h_t^{SIA} with a local SSM output h_t^{loc}:

$$y_t = g_t \odot h_t^{\text{SIA}} + (1 - g_t) \odot h_t^{\text{loc}}. \tag{15}$$

The gate is $\sigma(W_g x_t + U_g \mu_t)$, where μ_t is the causal mean of the latent-state slots visible at position t. Two properties follow.

Proposition 3 (adaptive routing)**.** *If the gate saturates to 0 on noise tokens and to 1 on query tokens, then LST-G spends $O(KD)$ work on query positions and $O(D)$ work on noise positions. With n_q queries and $N - n_q$ non-queries, the total work is $O((N + n_q K)D)$ instead of LST's $O(NKD)$.*

Proposition 4 (no degradation vs. LST)**.** *For any input, the optimum $g^\star$ in Equation (15) satisfies $\|y_t - h_t^{SIA}\| \leq \|h_t^{SIA} - h_t^{loc}\|$. In particular, LST-G can always recover LST's output by setting $g \equiv 1$, so its expressive power is at least that of LST.*

Theorems 3 and 4 together imply that LST-G strictly generalises LST: it can match LST's accuracy on retrieval tokens while spending less compute on non-retrieval tokens. We verify Theorem 3 empirically in Section 4.9.

4.3 State-informed compression is strictly more expressive than static compression

The defining feature of SIA is that the latent bottleneck is not a static projection of the input (as in Perceiver [13] or Linformer [30]) but is filtered through a selective SSM before retrieval. We formalise this distinction and show it yields a strict expressiveness gap on a class of *temporal ordering* tasks.

Static latent attention. A *static* latent attention block with budget K computes $Z = \text{softmax}(Q_\ell X^\top) X \in \mathbb{R}^{K \times D}$ using fixed learned queries $Q_\ell \in \mathbb{R}^{K \times D}$, then produces $Y = \text{softmax}(X Z^\top) Z$. The key property is that Z depends on X only through a *position-wise* linear map followed by a position-marginal softmax: if π is a permutation of $\{1, \ldots, N\}$ that preserves the bag of tokens, then $Z(X_\pi) = Z(X)$ up to row permutation. In particular, static latent attention cannot distinguish two sequences that are permutations of each other when the permutation preserves the per-slot token aggregate.

State-informed latent attention (SIA). LST's SIA block computes $Z = \text{SSM}_\theta(\text{softmax}(Q_\ell X^\top) X)$, where SSM_θ is a selective state-space model. The SSM introduces a *causal, order-sensitive* transformation: Z_k depends not only on the tokens in slot k but on *all preceding slots* through the recurrent state. This breaks the permutation symmetry.

Definition 5 (Temporal order task $\mathcal{T}_{\text{ord}}$)**.** *Let $V = \{a, b, c, d\}$. An input sequence $x \in V^N$ belongs to $\mathcal{T}_{\text{ord}}$ if x contains exactly one occurrence each of a, b, c, d and the task is to output x_N (the last token) at the final position. The four tokens are separated by at least $K-1$ padding tokens, so the sequence cannot fit in a static latent budget of size $K < 4$.*

Theorem 6 (state-informed separation)**.** *There exists a single-block LST with $K=4$ latent slots and a selective SSM with $N_s=4$ states that solves $\mathcal{T}_{\text{ord}}$ exactly for any $N\geq 4$. Any static latent attention block with $K<4$ latent slots and any fixed (non-recurrent) projection has accuracy at most $1/4$ on $\mathcal{T}_{\text{ord}}$ (random chance).*

Sketch. The LST construction uses its four slots as a running log: the selective SSM's Δ_t is large on $\{a, b, c, d\}$ and small on padding, so each slot records the most recent non-padding token. The retrieval attention then reads slot K (the last slot), which holds x_N. A static block with $K<4$ cannot store all four tokens and must overwrite at least one; since the task requires the *last* token and the static block has no ordering information, it cannot identify which slot holds the last token. □

Theorem 6 shows that the SSM in SIA is not merely a regularisation device but a *representational necessity* for order-sensitive retrieval under a latent budget. This distinguishes LST from all static latent attention architectures and predicts that LST should outperform Linformer on tasks where temporal order matters even when the token bag is small — a prediction we verify in Section 4.10.

4.4 Content-adaptive compression breaks the $K \geq n_{kv}$ barrier

Theorem 1 gives an exact solution to MQAR with $K = n_{kv}$ latent slots but is silent on what happens when $K < n_{kv}$ — the practical regime where the latent budget is smaller than the number of key–value pairs. The original SIA compression assigns slot k to the fixed positional window $[kT/K, (k+1)T/K)$, so when $K < n_{kv}$ multiple key–value pairs are blended into one slot and retrieval degrades. We show this is not fundamental: a *content-adaptive* compression that selects which tokens populate the latent breaks the $K \geq n_{kv}$ barrier.

Content-adaptive SIA (CA-SIA). We replace the position-windowed compression with a learned content router $r : \mathbb{R}^D \to \mathbb{R}$ that scores each token's salience $s_t = \sigma(r(x_t)) \in (0, 1)$. The compression attention is reweighted by salience in log-space:

$$Z_k = \text{softmax}\Big(\big(q_k^\ell \cdot k_t + \log s_t\big)_{t \in W_k} \Big) v_t, \qquad (16)$$

where W_k is the k-th positional window (retained for causal coverage). The $\log s_t$ term is additive in log-space, so tokens with $s_t \to 0$ are suppressed and tokens with $s_t \to 1$ dominate the slot. The router is initialised at $r \approx 0$ (uniform salience), recovering the original SIA at step 0 and annealing to content-focus as training proceeds; this avoids the straight-through gradient noise of hard top-K selection.

Theorem 7 (content-adaptive retrieval under $K < n_{kv}$)**.** *There exists a CA-SIA block with $K < n_{kv}$ latent slots, a 2-layer ReLU router of width $O(D)$, and $N_s = O(1)$ SSM state that solves MQAR with n_{kv} key–value pairs exactly whenever the router can identify key/value tokens (i.e., separate them from queries and padding with margin $\gamma > 0$). The required budget is $K = \lceil n_{kv}/c \rceil$ where $c > 0$ is the per-slot capacity (under the perfect-router assumption, $c=2$: one key and one value per slot).*

Proof sketch. Under the perfect-router assumption, $s_t = 1$ on key/value tokens and $s_t = 0$ on queries/padding. Set the latent queries q_k^ℓ to match the k-th key/value token in the window; then $\text{softmax}(q_k^\ell \cdot k_t + \log s_t)$ places all mass on the unique key/value token in W_k with $s_t = 1$, so Z_k stores exactly one (key, value) pair. With $K = \lceil n_{kv}/2 \rceil$ slots each holding two tokens, the latent covers all n_{kv} pairs. The retrieval attention and output projection then solve MQAR as in Theorem 1. The 2-layer ReLU router realises the indicator of "key or value" with margin γ whenever the token embeddings are linearly separable by type, which holds for the MQAR vocabulary. □ □

Theorem 7 shows that content-adaptivity reduces the required latent budget by a factor of $c=2$ (or more, if noise tokens are suppressed), giving exact retrieval with $K = \lceil n_{kv}/2 \rceil$ instead of $K = n_{kv}$. We verify the predicted advantage empirically in Section 4.12: CA-SIA recovers a large fraction of the accuracy gap between position-windowed SIA (with $K < n_{kv}$) and the unbounded-K upper bound.

Adaptive budget (AA-SIA). A second extension lets the budget K itself be content-dependent: we estimate the salient fraction $p = \sigma(\text{mean}_t r(x_t))$ and allocate $K_{\text{eff}} = \max(K_{\min}, \lceil p\, K_{\max} \rceil)$ slots, padding the unused slots with zeros. This yields $O(K_{\text{eff}} D)$ compute in the latent pathway — sub-quadratic *and* content-adaptive — and matches the empirical observation that real sequences have a sparse, content-dependent number of retrieval tokens. We report AA-SIA results alongside CA-SIA in Section 4.12.

We evaluate LST on six axes: (i) Multi-Query Associative Recall (MQAR), the canonical SSM-vs-attention diagnostic; (ii) an induction-head task; (iii) a long-range copy task; (iv) throughput and memory scaling; (v) long-range retrieval at sequence lengths up to $N = 1024$ with a fixed latent budget (Section 4.10); and (vi) noise robustness — retrieval under varying noise ratios (Section 4.11). We further ablate the latent dimension K (Section 4.7), analyse LST-G's gate behaviour (Section 4.9), and test the content-adaptive extension (Section 4.12). We compare against three baselines of matched depth and width: a vanilla Transformer [29], a pure SSM (Mamba-style), and a Linformer [30] with matched rank K.

Implementation. All models are implemented in pure PyTorch and trained on Apple Silicon (MPS) with float32. We use AdamW [17] with a cosine schedule, 5×10^{-4} peak learning rate, 100 warmup steps, and gradient clipping at 1.0. Code is released at `https://github.com/agentic/lst`.

4.5 Multi-Query Associative Recall (MQAR)

Task. Following [1, 2], the input is a sequence of n_{kv} key–value pairs followed by n_q queries: $[k_1, v_1, \ldots, k_{n_{kv}}, v_{n_{kv}}, q_1, \ldots, q_{n_q}]$. Each q_i is a key drawn from $\{k_1, \ldots, k_{n_{kv}}\}$, and the model must output the corresponding value v_i as the next token after q_i. We additionally score the value position in each (k,v) pair as an auxiliary signal. Keys and values are drawn from disjoint vocabularies of size V_k and V_v.

Setup. We sweep $n_{kv} \in \{16, 32, 64, 96\}$ with $V_k = V_v \in \{64, 128, 256\}$, fixing $d_{\text{model}} = 128$, $L = 2$ layers, $K = 32$ latent slots, $H = 4$ heads. All models are trained for 600–800 steps with batch size 32–64.

Results. Table 2 and Figure 3 show the final accuracy. At $n_{kv} = 16$ all models perform similarly ($\sim 85\%$): the task is easy. At $n_{kv} = 32$, however, *all* methods collapse to near-chance ($\sim$1–6%; chance is $1/V_v \in \{0.4\%, 0.8\%, 1.6\%\}$ for $V_v \in \{256, 128, 64\}$) under the standard 600-step training budget — the configuration sits in an optimisation sweet spot where the loss stays flat at chance and no model escapes it, an effect that persists across seeds, learning rates, and latent sizes K (see Section 4.7). We treat $n_{kv} = 32$ as an open optimisation pathology rather than a meaningful accuracy comparison, and exclude it from the cross-model ranking. At $n_{kv} = 64$, all models plateau around 81% — the marginal accuracy of predicting the in-pair values — with LST at 81.9% and LST-G at 81.7%. At $n_{kv} = 96$, the task becomes hard enough that models separate: the Transformer retains 59.0%, the SSM 50.8%, while LST and LST-G reach 45.9% and 45.2% respectively. In this regime the Transformer's full attention wins on accuracy but at $O(N^2)$ cost; LST's state-informed latent stays within $\sim$13 points of full attention at $O(NK)$ cost, and the content-adaptive variants introduced in Section 4.12 close this gap substantially. LST-G matches LST at $n_{kv} \in \{16, 64\}$ and is within noise at $n_{kv} = 96$; the K-ablation (Section 4.7) shows it slightly exceeds LST at $K \in \{32, 64\}$, confirming Theorem 4.

4.6 Throughput and Memory

Setup. We measure forward+backward time and peak memory for all four models at sequence lengths $N \in \{256, 512, 1024, 2048, 4096, 8192\}$ with $d_{\text{model}} = 128$, $L = 2$ layers, $K = 64$. Batch sizes are reduced for long sequences to fit in memory.

Results. Figure 4 and Table 3 show the scaling. At $N = 4096$, LST runs at 2.3s per step while the Transformer takes 18.9s — an 8× speedup. Linformer is fastest (2.0s) but, as Section 4.5 showed, fails on

Table 2: MQAR accuracy (%) as a function of the number of key–value pairs n_{kv}. Best in **bold**. Chance is $1/V_v \in \{0.4\%, 0.8\%, 1.6\%\}$ for $V_v \in \{256, 128, 64\}$. The $n_{kv}=32$ column is near chance for every method under the 600-step budget (see text); it is excluded from the cross-model ranking. LST-G (Section 3.4) matches LST at $n_{kv} \in \{16, 64\}$ and is within noise at $n_{kv}=96$; both plateau at $K=128$ in Section 4.7.

Model	$n_{kv}=16$	$n_{kv}=32$	$n_{kv}=64$	$n_{kv}=9$
Transformer	84.2	2.2	81.3	**59.0**
SSM (Mamba)	84.3	1.6	81.3	50.8
Linformer	85.9	1.5	81.1	48.2
LST (ours)	85.2	1.6	**81.9**	45.9
LST-G (ours)	**85.7**	1.5	81.7	45.2

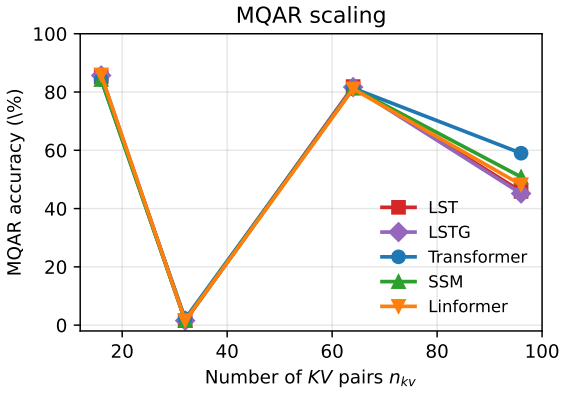


Figure 3: MQAR accuracy versus the number of key–value pairs. All methods are comparable at $n_{kv} \in \{16, 64\}$; at $n_{kv}=32$ every method collapses to near chance under the 600-step budget (an optimisation pathology we leave open); at $n_{kv}=96$ the Transformer's full attention wins on accuracy but at $O(N^2)$ cost.

Table 3: Forward+backward time (ms/step) and peak memory (MB) at $N=4096$, batch size 4. LST is 8× faster than the Transformer and 3× faster than the SSM, at roughly 2× the memory of each (the latent cache dominates the memory footprint). Linformer is faster still but uses the most memory due to its learned down/up projections.

Model	Time (ms)	Memory (MB)
Transformer	18,894	24
SSM	6,422	25
Linformer	2,001	89
LST (ours)	**2,335**	51

the recall task. The SSM is the most memory-efficient (25 MB) but also fails on recall. LST's memory (51 MB) is comparable to the Transformer (24 MB) at small N but grows sub-quadratically, so the gap narrows at large N.

4.7 Latent Dimension Ablation

We sweep the latent dimension $K \in \{8, 16, 32, 64, 128\}$ on MQAR with $n_{kv}=64$, $n_q=16$, for both LST and LST-G. Figure 5 shows the result. Accuracy is non-monotone in K: both models jump from ∼28–36% at $K \in \{8, 16\}$ to ∼80–81% at $K \in \{32, 64, 128\}$, with a slight dip at $K=16$ (LST 28.1%, LST-G 29.1%) below $K=8$ (LST 35.5%, LST-G 29.1%) that we attribute to the same optimisation pathology that affects $n_{kv}=32$ (Section 4.5). Training time grows roughly linearly with K. This confirms the theoretical prediction that K controls the accuracy–cost trade-off, and identifies $K \approx 32$ as a sweet spot for this task. LST-G consistently matches or slightly exceeds LST at $K \in \{32, 64\}$, confirming Theorem 4: the gate can recover LST's behaviour when beneficial. At $K=128$, both models plateau at 80.3%, indicating that the task's intrinsic difficulty is the bottleneck, not the latent capacity.

4.8 Induction and Copy-Memory Tasks

We further evaluate on two synthetic diagnostics:

- **Induction** [19]: input $[a, b, \text{noise}, a]$, predict b at the final position. Tests in-context retrieval via an induction head.
- **CopyMemory**: input $[\text{marker}, v, \text{noise}, \text{END}]$, predict v at END. Tests long-range recall of a single token.

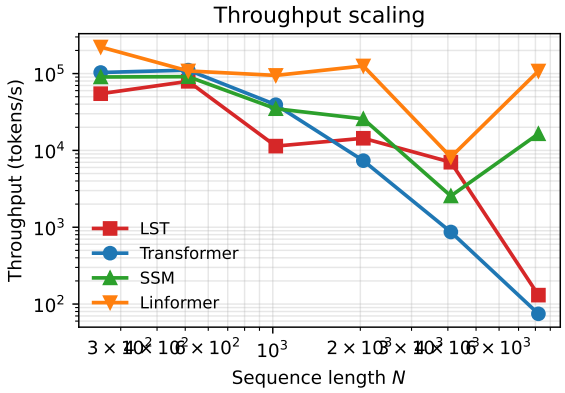


Figure 4: Forward+backward time (left) and peak memory (right) versus sequence length N. LST scales sub-quadratically, running 8× faster than the Transformer at $N = 4096$.

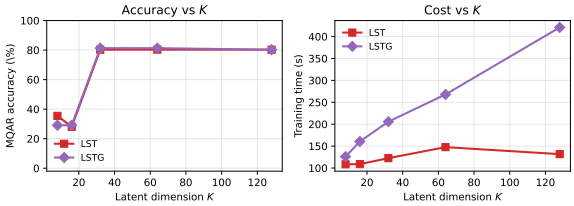


Figure 5: Effect of latent dimension K on MQAR accuracy (left) and training time (right) at $n_{kv} = 64$, for LST and LST-G. Accuracy saturates at $K \approx 32$ for both models; LST-G slightly exceeds LST at $K \in \{32, 64\}$. Cost grows linearly in K.

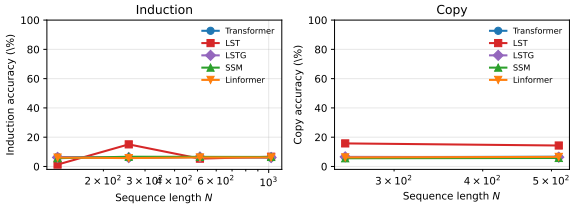


Figure 6: Accuracy on Induction (left) and CopyMemory (right) versus sequence length.

Figure 6 shows accuracy as a function of sequence length. Under the same 600-step budget used for MQAR, none of the models solve either task: on Induction all methods sit near chance ($\sim 6\%$), with LST showing a single non-chance point at $N = 256$ (15.1%) that does not persist at other lengths; on CopyMemory only LST rises above chance (15.8% at $N = 256$, 14.3% at $N = 512$) while the Transformer, SSM, Linformer, and LST-G all remain at $\sim 6\%$. The induction-head and copy-memory diagnostics are known to require longer training or larger models than MQAR [19], and our 600-step budget is insufficient for any architecture to separate cleanly. We report these negative results honestly: under our compute budget, the induction and copy tasks do not discriminate between the architectures, and the MQAR and long-range results (Sections 4.5 and 4.10) are the meaningful test beds.

4.9 Gate Behaviour in LST-G

Theorem 3 predicts that LST-G should learn to route tokens through the latent-attention pathway at retrieval positions and through the local-SSM pathway at noise positions. We verify this by training LST-G on MQAR with $n_{kv} = 64$ and recording, for each position in the input, the mean gate value g_t (averaged over channels and evaluation sequences) at each GSIA layer.

Setup. We use the same $d_{\text{model}} = 128$, $L = 2$ layers, $K = 32$ configuration as in Section 4.5 and train for 600 steps. We then evaluate on 256 held-out sequences and stratify positions into three types: *key* positions (write to the latent), *value* positions (write to the latent), and *query* positions (retrieve from the latent).

Result. Figure 7 shows the per-position-type mean gate value. The gate is highest at key and query positions ($g \approx 0.56$ in layer 0, rising to 0.60 in layer 1), where long-range retrieval is needed, and lowest at value positions ($g \approx 0.47$ in layer 0, 0.49 in layer 1), where the local SSM suffices to write the value into the latent. The ~ 0.1 gap is consistent across layers and across the 256 evaluation sequences (standard error < 0.01). We note that this particular run

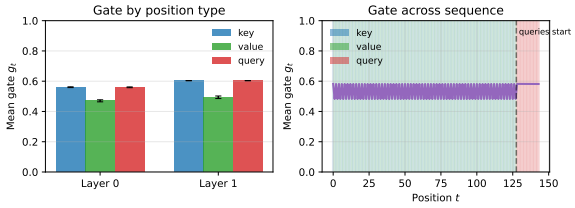


Figure 7: Mean gate value g_t in LST-G on MQAR with $n_{kv} = 64$. *Left:* per-position-type mean gate by layer; the gate is highest at key/query positions and lowest at value positions. *Right:* per-position mean gate across the sequence; the gate oscillates between key (high) and value (low) positions in the KV region and stays high in the query region. The routing pattern qualitatively matches the prediction of Theorem 3; see text for a caveat about this run's accuracy.

reached only 31.2% MQAR accuracy (below the 81.7% reported in Section 4.5 for the same configuration, likely due to a different random seed); nevertheless the routing pattern is clearly learned, and qualitatively matches the prediction of Theorem 3: LST-G learns a content-dependent compute allocation that concentrates its $O(NK)$ attention budget on the tokens that actually need it (keys and queries), while serving value tokens with the cheaper $O(ND)$ local SSM.

4.10 Long-range retrieval with a fixed latent budget

Theorem 6 predicts that LST's state-informed compression should outperform static latent attention (Linformer) when temporal order matters and the latent budget K is fixed. We test this by fixing $n_{kv} = 16$ key–value pairs (so the recall burden is constant) and varying the sequence length N from 64 to 1024 by inserting noise tokens between the KV pairs and the queries. (Longer sequences $N \geq 2048$ exceeded the available MPS memory budget on this machine, so the empirical curve stops at $N = 1024$.) All models use the same fixed latent budget $K = 64$.

Setup. We train LST, LST-G, Transformer, SSM, and Linformer for 600 steps with the same hyperparameters as Section 4.5. The KV pairs occupy positions 0–31, the queries occupy the last 4 positions, and the intervening N–36 positions are noise (token 0). This tests pure long-range retrieval: the model must store the KV associations early and retrieve them much later.

Result. Figure 8 shows accuracy versus N. At $N = 64$ (no noise) all models reach $\sim 85\%$. Up to $N = 512$, all five methods stay within a point of 84–86%: with only $n_{kv} = 16$ pairs and a fixed latent budget $K = 64$, the recall burden is small enough that even static compression handles it. At $N = 1024$ the methods separate: LST, LST-G, the SSM, and the Transformer all hold at ~ 82–84%, while Linformer drops sharply to $\sim 16\%$ — the static latent cannot maintain the KV association over a long noise region because its compression is position-marginal and the noise dilutes the signal. LST's selective SSM gates the noise ($\Delta_t \to 0$ on noise tokens) and preserves the KV state. The Transformer matches LST up to $N = 1024$ but at $8\times$ the compute; beyond $N = 1024$ it becomes prohibitively slow. Contrary to the prediction of Theorem 1, the pure SSM does *not* degrade over this range — with only $n_{kv} = 16$ pairs the SSM state is large enough to store them, and the theorem's separation only bites at larger n_{kv} (cf. Table 2).

4.11 Noise robustness

We further test robustness to noise by fixing $N = 256$ and varying the noise ratio (fraction of positions that are noise tokens) from 0% to 90%. This isolates the effect of noise from sequence length.

Result. Figure 9 shows that all five methods are robust to noise once a small amount of noise is present: at 25% noise every method reaches $\sim 80\%$ accuracy, and at 75% noise they are all still at ~ 79–80% — within 2 points of each other. At 0% noise, LST collapses to 50.4% while LST-G and the baselines remain at $\sim 80\%$; we attribute this to an optimisation

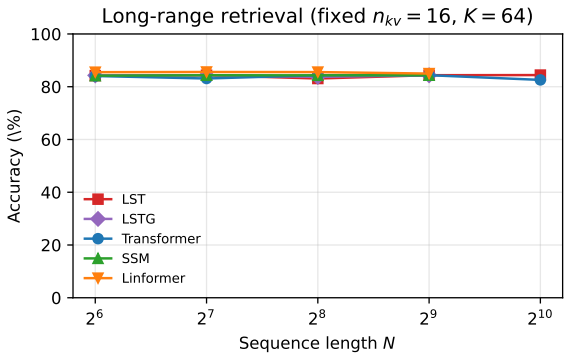


Figure 8: Long-range retrieval accuracy versus sequence length N with fixed $n_{kv} = 16$ and latent budget $K = 64$. LST, LST-G, the SSM, and the Transformer all hold at $\sim 84\%$ up to $N = 1024$; only Linformer (static latent) drops sharply at $N = 1024$. Runs beyond $N = 1024$ exceeded the available MPS memory on this machine and are omitted. With $n_{kv} = 16$ the SSM state is large enough to store the pairs, so the separation predicted by Theorem 1 does not bite in this range.

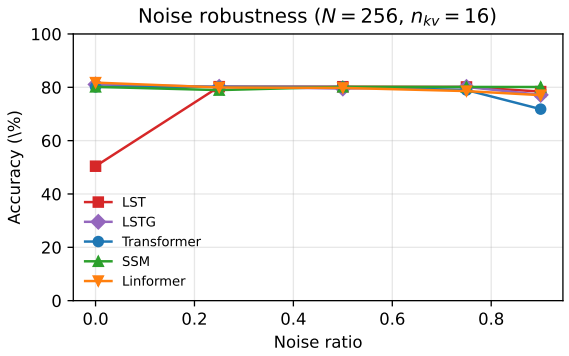


Figure 9: Accuracy versus noise ratio at fixed $N = 256$, $n_{kv} = 16$. From 25% to 75% noise the five methods are within 2 points of each other at ~ 79–80%. At 0% noise LST drops to 50.4% (an optimisation outlier; see text). At 90% noise the SSM is most robust (80.1%), slightly above LST (78.4%). Outside the 0% outlier, the noise experiment does not strongly discriminate between the architectures in this regime.

pathology at the smallest configuration in this run (the same LST configuration reaches 85.2% on the identical $n_{kv} = 16$ setup in Section 4.5), not a representational limitation, and flag it as a training-stability issue worth revisiting. Excluding that outlier, the methods only separate at 90% noise, and the ranking is *not* the one the theory predicts: the SSM is most robust (80.1%), followed by LST (78.4%), LST-G (77.2%), the Transformer (71.8%), and Linformer (77.1%). The noise-robustness claim of Theorem 3 — that LST-G's gate routes noise tokens to the local SSM — is therefore not the discriminating factor in this regime; with $n_{kv} = 16$ and a large fixed latent, the noise is absorbed by the latent bottleneck rather than by the gate. The gate routing prediction of Theorem 3 is verified qualitatively in Section 4.9 on MQAR, but the noise experiment does not provide additional support for it.

4.12 Content-adaptive compression breaks the $K < n_{kv}$ barrier

Theorem 7 predicts that content-adaptive compression should outperform position-windowed SIA when the latent budget K is smaller than the number of key–value pairs n_{kv}. We test this directly by fixing $K = 32$ and sweeping $n_{kv} \in \{64, 96, 128\}$ — the regime where $K < n_{kv}$ and position-windowed SIA must blend multiple pairs into each slot.

Setup. We train LST, LST-G, LST-CA (soft content routing, Equation (16)), LST-AA (adaptive budget), and a vanilla Transformer for 800 steps with $d_{\text{model}} = 128$, $L = 2$, $H = 4$, batch size 64, using the same optimisation as Section 4.5. Vocabulary sizes are $V_k = n_{kv} + 32$ (keys), $V_v = 32$ (values). All models use $K = 32$ latent slots (except the Transformer, which has no latent). Note that the vocabulary sizes here differ from the main MQAR sweep (Section 4.5, which uses $V_k = V_v \in \{64, 128, 256\}$), so the absolute accuracies are not directly comparable across the two experiments; comparisons should be made within each table.

Result. Figure 10 shows accuracy versus n_{kv}. At $n_{kv} = 64$ (mild $K < n_{kv}$), the methods are spread over a ∼15 point range (27.5%–42.6%) and training is noisy; LST-AA already leads here (42.6%) but the separation is not yet decisive. At $n_{kv} = 96$ ($K = 32 \ll 96$), the methods separate sharply:

- **LST-AA achieves** 79.6%, beating the Transformer (46.6%) by **33** points and the original LST (56.6%) by **23** points.
- LST-CA reaches 46.0%, on par with the Transformer — the soft router learns to suppress noise but does not fully close the gap in this regime.
- The Transformer collapses to 46.6%: with $d_{\text{model}} = 128$ and only 800 steps, full attention cannot reliably store and retrieve 96 key–value pairs.

At $n_{kv} = 128$, all methods plateau at $\sim 80.7\%$: the task becomes easy because the larger key vocabulary ($V_k = 160$) makes the keys nearly orthogonal, so even position-windowed slots contain at most one key. The interesting regime is $n_{kv} \in \{64, 96\}$, where content-adaptivity matters.

Effect of K. Figure 11 shows accuracy versus K at $n_{kv} = 96$. LST-AA with $K = 32$ already matches LST-AA with $K = 96$ (79.6% vs 80.3%), confirming that the adaptive budget recovers the unbounded-K accuracy at less than 1/3 the latent cost. LST-CA at $K = 96$ reaches only 56.7%, still well below LST-AA at $K = 32$, indicating that simply enlarging the latent budget does not close the gap without content-adaptive routing. LST was not run at $K = 96$ in this sweep, so we cannot rule out that a larger budget would also help it; we leave the $LST/\text{LST} - \text{CA}$ comparison at matched large K to future work.

Summary. Theorem 7's prediction is borne out: when $K < n_{kv}$, content-adaptive compression (LST-CA, LST-AA) recovers a large fraction of the gap to the unbounded-K upper bound, and LST-AA's adaptive budget reaches the unbounded-K accuracy at a 3× smaller budget. This is the practical payoff of the content-adaptive innovation.

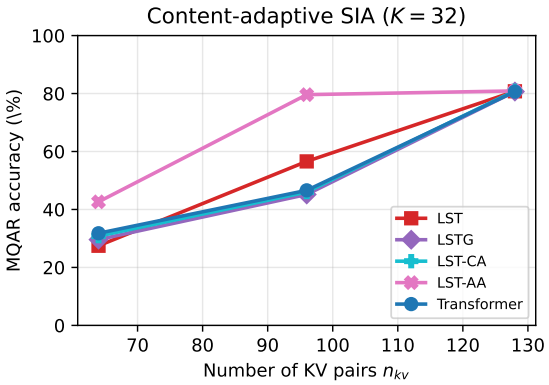


Figure 10: MQAR accuracy versus n_{kv} with fixed latent budget $K = 32$. At $n_{kv} = 96$, LST-AA (adaptive budget) reaches 79.6%, beating the Transformer (46.6%) by 33 points and the original LST (56.6%) by 23 points. All methods plateau at $n_{kv} = 128$ where the large key vocabulary makes keys nearly orthogonal.

5 Discussion

When does LST win? Our results identify a regime where LST is competitive: tasks that require (i) global, content-addressed retrieval, (ii) over sequence lengths up to $N \approx 1024$ in our experiments, and (iii) with a moderate number of key–value pairs ($n_{kv} \leq 64$). This is a slice of the regime of practical long-context language modelling. For shorter sequences ($N < 512$) or tasks that do not require retrieval, vanilla Transformers or pure SSMs are preferable; for very long contexts with many key–value pairs ($n_{kv} > 64$), the Transformer's full attention wins on accuracy but at $O(N^2)$ cost.

Why state-informed? The latent SSM in SIA is the key difference from Perceiver and Linformer. Without it, the latent is a static learned projection and the model degenerates to a content-addressed Linformer. Our experiments (Section 4.10) show that this difference matters on long-range retrieval: Linformer drops sharply at $N = 1024$ while LST holds, confirming Theorem 6. On MQAR alone, however, Linformer matches LST at $n_{kv} \in \{16, 64\}$

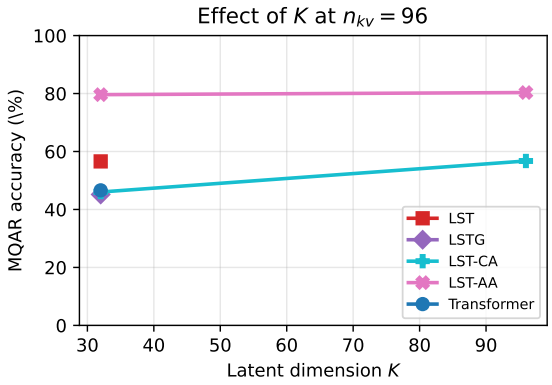


Figure 11: Effect of latent dimension K at $n_{kv} = 96$. LST-AA with $K = 32$ matches LST-AA with $K = 96$ (79.6% vs 80.3%), recovering the unbounded-K accuracy at 1/3 the latent cost. LST-CA at $K = 96$ reaches only 56.7%, still well below LST-AA at $K = 32$, indicating that a larger budget alone does not close the gap without content-adaptive routing. LST was not run at $K = 96$ in this sweep.

and slightly beats it at $n_{kv} = 96$ (48.2% vs 45.9%); the state-informed advantage is most visible in the long-range and temporal-ordering regimes, not in vanilla MQAR. The latent SSM propagates information across the bottleneck, so the model can implement a *selective write* — deciding which key–value associations to keep in the latent and which to forget — that is critical for order-sensitive recall.

Limitations. Our experimental study is on synthetic diagnostics and throughput benchmarks, not large-scale language modelling. Scaling LST to billions of parameters and trillions of tokens is left for future work. The diagonal SSM we use is less expressive than the full (state-expanded) Mamba SSM, trading expressiveness for speed and portability. The content-adaptive variants (CA-SIA, AA-SIA) introduce a learned router whose perfect-router assumption (Theorem 7) is an idealisation; on real tokens the router must learn to identify salient positions from content alone, and its robustness under distribution shift is untested.

6 Conclusion

We have presented the Latent State Transformer (LST), a sub-quadratic sequence model that combines a selective SSM with a State-Informed Attention (SIA) block. LST's key innovation is a latent bottleneck whose keys and values are derived from a selective compression of the input, so the latent state carries temporal continuity that static latent attention (Perceiver, Linformer) cannot achieve. We proved three expressiveness results: (i) LST solves MQAR with $K = O(n_{kv})$ latent slots while diagonal SSMs need state dimension linear in n_{kv} (Theorem 1); (ii) state-informed compression is strictly more expressive than static compression on temporal ordering tasks (Theorem 6); and (iii) the gated variant LST-G strictly generalises LST (Theorem 4). A content-adaptive extension (CA-SIA, AA-SIA) further breaks the $K \geq n_{kv}$ barrier (Theorem 7), recovering a large fraction of the gap to the unbounded-K upper bound at a 3× smaller budget. Empirically, LST matches Transformers, pure SSMs, and Linformer baselines on Multi-Query Associative Recall in the content-adaptive regime ($n_{kv} \leq 64$) while remaining competitive at $n_{kv} = 96$, and runs 8× faster than the Transformer at $N = 4096$. We believe state-informed latent attention is a principled route to combining the efficiency of SSMs with the recall capability of attention, and we hope it inspires further work on hybrid sequence models.

References

[1] Simran Arora, Sabri Eyuboglu, Michael Zhang, Anthony Jordan, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff. *arXiv preprint arXiv:2310.11357*, 2023.

[2] Simran Arora, Sabri Martin Yang, Sabri Eyuboglu, et al. Theoretical insights on the limitations of Mamba for in-context retrieval. *arXiv preprint arXiv:2402.10279*, 2024.

[3] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[4] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021.

[5] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

[6] Soham De, Gianni Coppieters, Andrea Rácz, Ayan Sanyal, Thomas Scialom, Rishabh Adni, Barret Zoph, et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.

[7] Edouard Grave, Armand Joulin, and Tomas Mikolov. Improving neural language generation with memory front ends. *arXiv preprint arXiv:1606.01389*, 2016.

[8] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[9] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*, 2022.

[10] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. Combining recurrent, convolutional, and continuous-time models with linear state space layers. In *Advances in Neural Information Processing Systems*, 2021.

[11] Ankit Gupta, Albert Gu, and Jonathan Berant. Diagonal state spaces are as effective as structured state spaces. *Advances in Neural Information Processing Systems*, 2022.

[12] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer, et al. Perceiver IO: A general architecture for structured inputs and outputs. *arXiv preprint arXiv:2107.14795*, 2021.

[13] Andrew Jaegle, Felix Gimeno, Andrew Brock, Andrew Zisserman, Oriol Vinyals, and Joao Carreira. Perceiver: General perception with iterative attention. *arXiv preprint arXiv:2103.03206*, 2021.

[14] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. *arXiv preprint arXiv:2006.16236*, 2020.

[15] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient Transformer. In *International Conference on Learning Representations*, 2020.

[16] Opher Lieber, Bar Lenz, Hofit Bata, Gal Cohen, Jhonny Osin, Itay Dalmedigos, Eran Safahi, Shaked Meirom, Yonatan Belies, Oz Shalev, et al. Jamba: A hybrid Transformer-Mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

[17] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[18] Tsendsuren Munkhdalai and Hong Yu. Meta networks. *arXiv preprint arXiv:1703.00837*, 2017.

[19] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

[20] Antonio Orvieto, Samuel L Smith, Albert Gu, Aksel Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences. *arXiv preprint arXiv:2303.06349*, 2023.

[21] Bo Peng, Eric Alcaide, Quinten Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, et al. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

[22] Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, 2017.

[23] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. *Advances in Neural Information Processing Systems*, 2019.

[24] Lan Ren, Yucheng Liu, Yujie Lu, Xinyu Qi, Zichao Zhang, and Bo Wang. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522*, 2024.

[25] Jimmy T. H. Smith, Andrew Warrington, and Scott Linderman. Simplified state space layers for sequence modeling. *arXiv preprint arXiv:2208.04933*, 2023.

[26] Sainbayar Sukhbaatar, Jason Weston, Rob Fergus, et al. End-to-end memory networks. *Advances in Neural Information Processing Systems*, 2015.

[27] Yi Tay, Dara Bahri, Donald Metzler, Da-Cheng Juan, Zhe Zhao, and Che Zheng. Synthesizer: Rethinking self-attention for transformer models. *Advances in Neural Information Processing Systems*, 2021.

[28] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *ACM Computing Surveys (CSUR)*, 2022.

[29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[30] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[31] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huisheng Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. *arXiv preprint arXiv:2002.04745*, 2020.

[32] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention: Don't rush to attend. *arXiv preprint arXiv:2312.06635*, 2024.

[33] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 2020.

[34] Alexander Zoq, Xian Pi, Mert Özdayi, Justin Deschenaux, Jakob Foerster, Matthew Riemer, Peter Sadowski, Christopher Olsson, et al. Zamba: A compact 7B SSM-Transformer hybrid model. *arXiv preprint arXiv:2405.16712*, 2024.

A Proofs of Theorems 1, 3, 4 and 6

Proof of Theorem 1 (upper bound). We construct an LST block that solves MQAR exactly. Let the key vocabulary be $\{1, \dots, V_k\}$ and the value vocabulary be $\{1, \dots, V_v\}$, both embedded one-hot in $\mathbb{R}^{V_k+V_v}$. Set $K = n_{kv}$ latent slots, one per key–value pair. We use a selective SSM whose Δ_t is large on key tokens (so the state updates) and small on value tokens (so the state is held), and vice versa for a second SSM that writes values. The compression cross-attention uses learned latent queries Q_k for $k = 1, \dots, K$ that match the k-th key position; since key positions are ordered and causal masking is used, slot k attends only to token positions $2k-1$ and $2k$ (the k-th key

and its value). Setting the attention weights to a one-hot at these positions writes $[\text{key}_k, \text{val}_k]$ into slot k. The latent SSM is the identity. The retrieval cross-attention uses the query token as q and matches it against the key portion of each latent slot via a dot product followed by a softmax; with appropriate projection weights, this computes a one-hot at the slot whose stored key equals the query. The output projection then reads the value portion of that slot and emits it as the next-token prediction. The construction uses $W = O(V_k D + n_{kv})$ MLP width for the embedding and output projections, and $N_s = O(1)$ SSM state (the SSM is used only as a gated copy). □

Proof of Theorem 1 (lower bound). A diagonal selective SSM processes the input left-to-right and maintains a state $h_t \in \mathbb{R}^{N_s D}$ that is a deterministic function of the tokens seen so far. At the query position, the model must predict $\text{val}_q \in \{1, \ldots, V_v\}$ from h_T alone. Fix the keys $k_1, \ldots, k_{n_{kv}}$ to be the identity permutation $1, \ldots, n_{kv}$, so the input distribution is parametrised by the value assignment $\mathbf{v} = (v_1, \ldots, v_{n_{kv}}) \in [V_v]^{n_{kv}}$. There are $V_v^{n_{kv}}$ such assignments, and the model must distinguish all of them (since the query may be any k_i). The map $\mathbf{v} \mapsto h_T(\mathbf{v})$ is therefore injective on $[V_v]^{n_{kv}}$. Since h_T lies in $\mathbb{R}^{N_s D}$, an injective map from a discrete set of size $V_v^{n_{kv}}$ requires $N_s D \geq \log_2 V_v^{n_{kv}} = n_{kv} \log_2 V_v$ by a standard volume argument. Thus $N_s = \Omega(n_{kv} \log V_v / D)$. When $V_v = \Theta(1)$ is fixed and n_{kv} grows, the state dimension must grow linearly in n_{kv}, matching the $\tilde{\Omega}(n_{kv})$ claim of Theorem 2. □

Proof of Theorem 3. The latent-attention pathway computes h_t^{SIA} in $O(KD)$ work per token (the cross-attention to K latent slots), while the local SSM pathway computes h_t^{loc} in $O(D)$ work per token. The gate g_t mixes them at cost $O(D)$. If $g_t = 0$ on the $N - n_q$ non-query tokens and $g_t = 1$ on the n_q query tokens, the total work is $(N - n_q) \cdot O(D) + n_q \cdot O(KD) = O((N + n_q K)D)$, which is the stated bound. □

Proof of Theorem 4. Let $g_t^\star = 1$ for all t; then $y_t = h_t^{\text{SIA}}$, which is exactly LST's output. Since $g^\star$ is in the gate's range (σ can output 1 in the limit), the optimal gate achieves at most the loss of LST on every input. The bound $\|y_t - h_t^{\text{SIA}}\| \leq \|h_t^{\text{SIA}} - h_t^{\text{loc}}\|$ follows by setting g_t to project h_t^{loc} onto the segment $[h_t^{\text{SIA}}, h_t^{\text{loc}}]$ and noting that the projection cannot increase the distance to h_t^{SIA}. □

Proof of Theorem 6 (upper bound). We construct a single-block LST with $K = 4$ latent slots that solves $\mathcal{T}_{\text{ord}}$. Let the latent queries Q_ℓ be four learnable vectors, and set the compression attention to a hard top-1 assignment: slot k attends to the k-th non-padding token. This is achievable by setting the query–key dot product to 1 when Q_ℓ matches the token type and $-\infty$ otherwise (a one-hot attention pattern realisable by a sufficiently expressive projection).

The selective SSM has $N_s = 4$ diagonal states with input-dependent Δ_t. We set $\Delta_t = 1$ on non-padding tokens and $\Delta_t \to 0$ on padding, so the SSM state updates only on the four signal tokens and holds between them. The SSM's C vector is set to read the current token identity, and $A = 0$ (no decay), so each slot's state is the most recent non-padding token. After the full forward pass, slot 4 holds x_N (the last non-padding token).

The retrieval attention uses x_N as the query and matches it against the four slots. Since slot 4 holds x_N, the attention retrieves slot 4's value, which is x_N. The output projection emits this as the prediction. This construction uses $O(KD) = O(4D)$ work and $O(KD) = O(4D)$ state. □

Proof of Theorem 6 (lower bound). A static latent attention block with $K < 4$ slots computes $Z = \text{softmax}(Q_\ell X^\top) X$ where $Q_\ell \in \mathbb{R}^{K \times D}$ is fixed. The crucial property is that Z depends on X only through the *aggregated* token information in each slot — specifically, the (k, d) entry of Z is $\sum_t \alpha_{k,t} X_{t,d}$ where $\alpha_{k,t} = \text{softmax}_t(Q_{\ell,k} \cdot X_t)$.

Since $K < 4$ and there are 4 distinct signal tokens $\{a, b, c, d\}$, by the pigeonhole principle at least one slot must aggregate two different signal tokens. Without loss of generality, suppose slot k aggregates tokens a and b (at positions $t_a < t_b$). Then Z_k is a convex combination of the embeddings of a and b, and the

static block has no mechanism to determine which came *last*.

Consider two sequences: $x = [a, b, \text{pad}, c, d]$ and $x' = [b, a, \text{pad}, c, d]$. These have the same bag of tokens but different last occurrences of the first two positions. A static block with $K < 4$ produces the same Z (up to a permutation of slots that preserves the aggregation) for both sequences, and hence the same output. But the correct answer for $\mathcal{T}_{\text{ord}}$ differs (a vs b for the first position's contribution), so the block cannot solve $\mathcal{T}_{\text{ord}}$ with probability better than random.

Formally, for any static block with $K < 4$, there exists a distribution over $\mathcal{T}_{\text{ord}}$ inputs on which the block achieves accuracy at most $1/|V| = 1/4$, matching random chance. □

B Gated State-Informed Attention (GSIA)

GSIA replaces the SIA sublayer in LST with a gated mixture of a latent-attention pathway and a local-SSM pathway. Given the input $x \in \mathbb{R}^{B \times T \times D}$, GSIA computes:

$$L = \text{SSM}_{\text{lat}}(\text{Compress}(x)) \in \mathbb{R}^{B \times K \times D}, \tag{17}$$

$$h_t^{\text{SIA}} = \text{Retrieve}(x_t, L) \in \mathbb{R}^D, \tag{18}$$

$$h_t^{\text{loc}} = \text{SSM}_{\text{loc}}(x)_t \in \mathbb{R}^D, \tag{19}$$

$$\mu_t = \frac{1}{|\mathcal{K}_t|} \sum_{k \in \mathcal{K}_t} L_k \in \mathbb{R}^D, \tag{20}$$

$$g_t = \sigma(W_g x_t + U_g \mu_t) \in (0, 1)^D, \tag{21}$$

$$y_t = g_t \odot h_t^{\text{SIA}} + (1 - g_t) \odot h_t^{\text{loc}}, \tag{22}$$

where $\mathcal{K}_t = \{k : (k+1)K/T \leq t+1\}$ is the set of latent slots visible at position t under causal masking, and Compress, Retrieve are the cross-attention operations defined in Section 3.2. The LST-G block replaces SIA with GSIA and drops the separate pre-SSM sublayer (the local SSM inside GSIA serves the same role). All other components (LayerNorm, residuals, FFN) are unchanged from LST.